VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

APRAMEYA KULKARNI (1BM24CS049)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road,Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by APRAMEYA KULKARNI **(1BM24CS049)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab Program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100
void calculateInDegree(int adj[MAX][MAX], int in_degree[MAX], int n) {
    for(int i=0;i<n;i++) {
        in_degree[i] = 0;
        for(intj=0;j<n; j++) {
            if(adj[j][i]==1) {
                in_degree[i]++;
            }
        }
    }
}

void topologicalSort(int adj[MAX][MAX], int n) {
    intin_degree[MAX];
    int queue[MAX];
    intfront=0,rear=0;
    inttopo_order[MAX];
    int count = 0;
    calculateInDegree(adj, in_degree, n);
    for(inti=0;i<n;i++) {
    if(in_degree[i]== 0) {
```

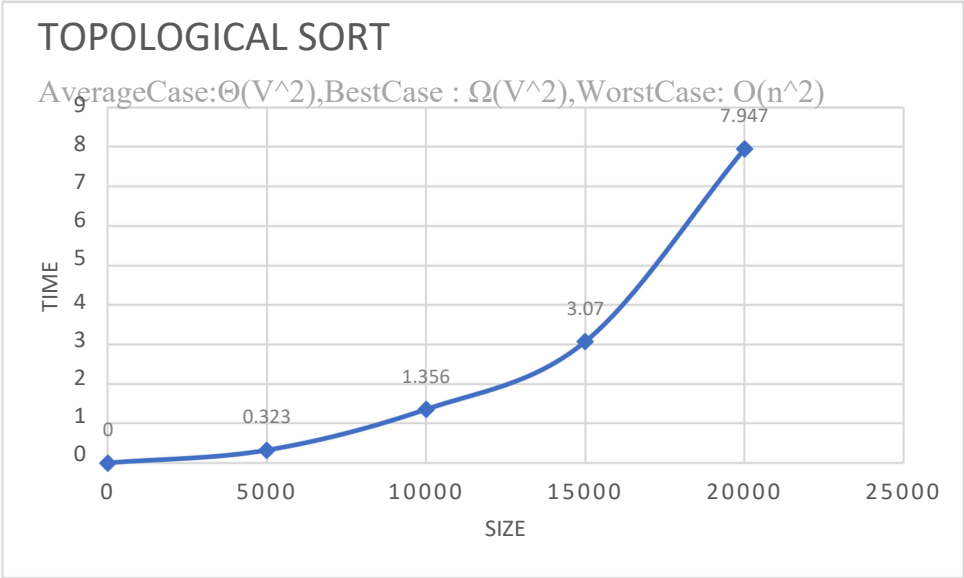
```
            queue[rear++] = i;
        }
    }

    while (front < rear) {
        int u = queue[front++];
        topo_order[count++] = u;
        for (int v = 0; v < n; v++) {
            if (adj[u][v] == 1) {
                in_degree[v]--;
                if (in_degree[v] == 0) {
                queue[rear++] = v;
                }
            }
        }
    }

    if (count < n) {
        printf("\nError:Graphcontainsa cycle. Topological ordering impossible.\n");
    } else {
        printf("\nTopologicalOrdering:\n");
        for (int i = 0; i < count; i++) {
            printf("%d ", topo_order[i]);
        }
        p r int f( "\ n") ;
    }
}
```

```
int main() {
int adj[MAX][MAX];
   int n, edges, u, v;

   printf("Enter the number of vertices: ");
   scanf("%d", &n);

   printf("Enter the adjacency matrix: ");
   for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
       scanf( "% d", &ad j[ i] [ j] ) ;
       }
   }
   topologicalSort(adj, n);
   return 0;
}
```

Output:

```
Enter the number of vertices: 7
Enter the adjacency matrix:
0 0 1 0 0 0 0
1 0 0 1 1 0 0
0 0 0 1 0 1 1
0 0 0 0 1 0 0
0 0 0 0 0 1 0
0 0 0 0 0 0 1
0 0 0 0 0 0 0
Topological order: 2 1 3 4 5 6 7
```

Graph:



LeetCode 207 – Course Schedule:

207. Course Schedule

Solved

Medium | Topics | Companies | Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a_i, b_i]` indicates that you **must** take course `b_i` first if you want to take course `a_i`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

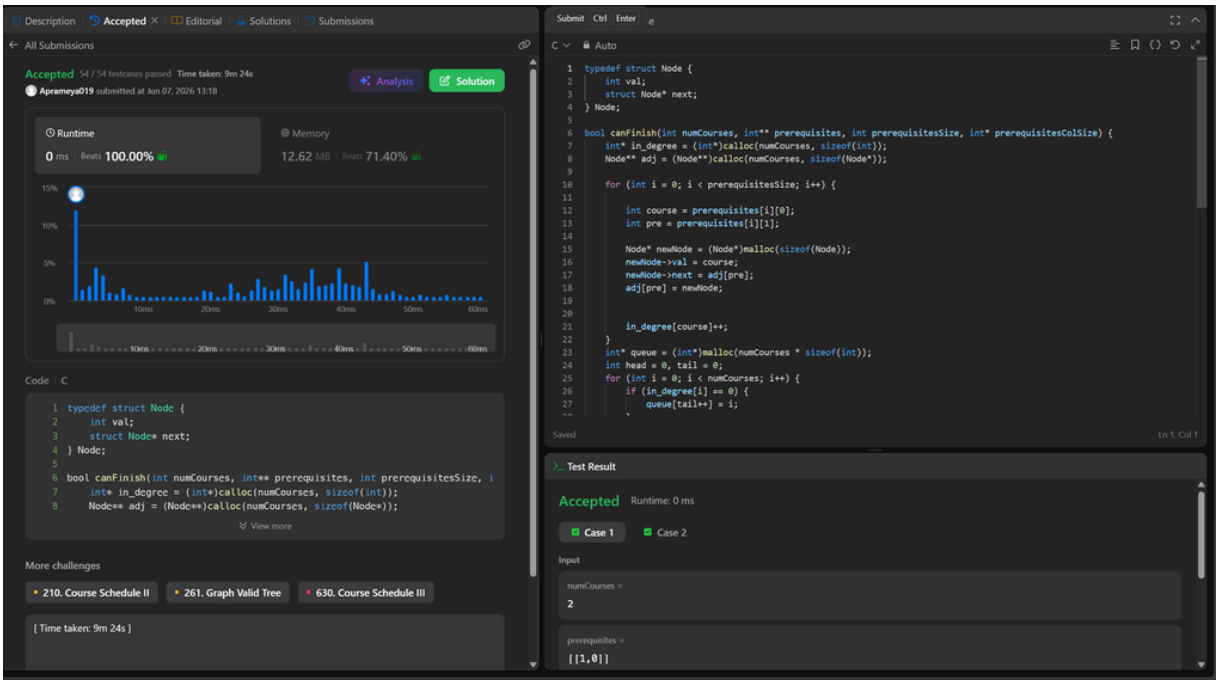
Code:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {

    int **graph = (int**)malloc(numCourses * sizeof(int*));
```

```
 int *graphSize = (int*) calloc(numCourses, sizeof(int*));
    int *indegree = (int*) calloc(numCourses, sizeof(int*));
    for(int i=0; i<numCourses; i++)
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
    for(int i=0; i<prerequisitesSize; i++){
        int courses = prerequisites[i][0];
        int prereq = prerequisites[i][1];
        graph[prereq][graphSize[prereq]++] = courses;
        indegree[courses]++;
    }
    int *queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear=0;
    for(int i=0; i<numCourses; i++){
        if(indegree[i]==0){
            queue[rear++] = i;
        }
    }
    int completed = 0;
    while(front<rear){
        int node = queue[front++];
        completed++;
        for(int i=0; i<graphSize[node]; i++){
            int neighbour = graph[node][i];
            indegree[neighbour]--;
            if(indegree[neighbour] == 0) queue[rear++] = neighbour;
        }
    }
    return completed == numCourses;
}
```

Output:


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
true
Expected
true


Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted
Analysis
Solution
Runtime
0 ms
Beats 100.00%
Memory
12.62 MB
Beats 71.40%
Code
C
typedef struct Node {
int val;
struct Node* next;
} Node;
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
int* in_degree = (int*)calloc(numCourses, sizeof(int));
Node** adj = (Node**)calloc(numCourses, sizeof(Node*));
for (int i = 0; i < prerequisitesSize; i++) {
int course = prerequisites[i][0];
int pre = prerequisites[i][1];
Node* newNode = (Node*)malloc(sizeof(Node));
newNode->val = course;
newNode->next = adj[pre];
adj[pre] = newNode;
in_degree[course]++;
}
int* queue = (int*)malloc(numCourses * sizeof(int));
int head = 0, tail = 0;
for (int i = 0; i < numCourses; i++) {
if (in_degree[i] == 0) {
queue[tail++] = i;
More challenges
210. Course Schedule II
261. Graph Valid Tree
630. Course Schedule III
[Time taken: 9m 24s]
Submit Ctrl Enter
Auto
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

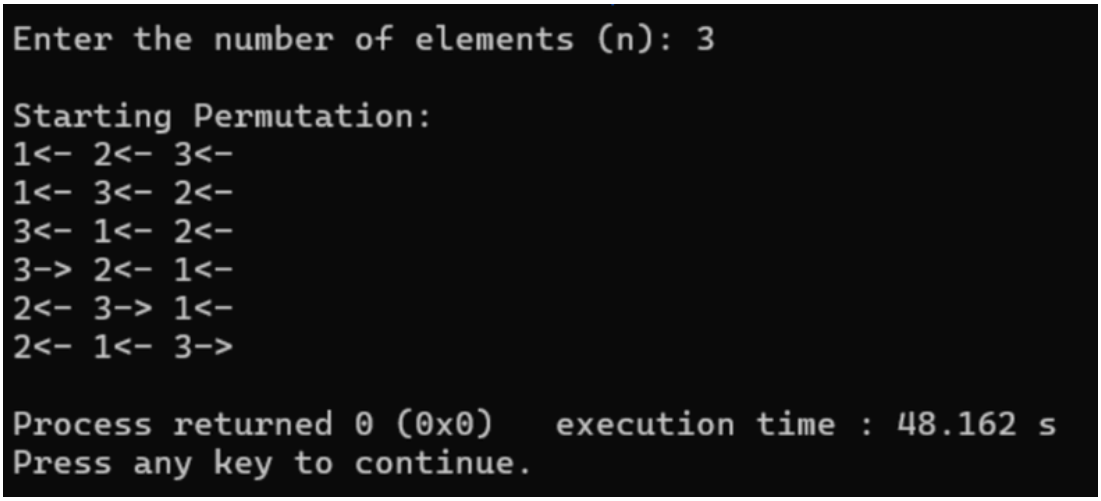
Code:

```
#include <stdio.h>

int getMobile(int a[], int dir[], int n)
{
   int mobile = 0, mobile_pos = -1;

   for(int i = 0; i < n; i++)
   {
      if(dir[i] == -1 && i != 0 && a[i] > a[i-1])   // Left
      {
         if(a[i] > mobile)
         {
            mobile = a[i];
            mobile_pos = i;
         }
      }

      if(dir[i] == 1 && i != n-1 && a[i] > a[i+1])  // Right
      {
         if(a[i] > mobile)
         {
            mobile = a[i];
            mobile_pos = i;
         }
      }
   }

   return mobile_pos;
}

void printPermutation(int a[], int n)
{
   for(int i = 0; i < n; i++)
      printf("%d ", a[i]);
   printf("\n");
}
```

```
int main()
{
 int n;

 printf("Enter n: ");
 scanf("%d", &n);

 int a[n], dir[n];

 // Initial permutation
 for(int i = 0; i < n; i++)
 {
 a[i] = i + 1;
 dir[i] = -1; // -1 = Left, 1 =
Right
 }

 printf("Permutations:\n");
 printPermutation(a, n);

 while(1)
 {
 int pos = getMobile(a, dir,
n);

 if(pos == -1)
 break;

 int mobile = a[pos];
```

```
// Swap mobile element
 if(dir[pos] == -1)
 {
 int temp = a[pos];
 a[pos] = a[pos-1];
 a[pos-1] = temp;
 temp = dir[pos];
 dir[pos] = dir[pos-1];
 dir[pos-1] = temp;
 pos--;
 }
 else
 {
 int temp = a[pos];
 a[pos] = a[pos+1];
 a[pos+1] = temp;
 temp = dir[pos];
 dir[pos] = dir[pos+1];
 dir[pos+1] = temp;
 pos++;
 }
 // Reverse directions of elements larger than mobile
 for(int i = 0; i < n; i++)
 {
 if(a[i] > mobile)
 dir[i] = -dir[i];
 }
 printPermutation(a, n);
 }
 return 0;
}
```

OUPUT:


Enter the number of elements (n): 3
Starting Permutation:
1<- 2<- 3<-
1<- 3<- 2<-
3<- 1<- 2<-
3-> 2<- 1<-
2<- 3-> 1<-
2<- 1<- 3->
Process returned 0 (0x0) execution time : 48.162 s
Press any key to continue.

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include <stdio.h>


void merge(int a[],int l, int m, int r) {
    int i, j, k;
    int n1=m-l+1;
    int n2=r-m;
    int L[n1], R[n2];
    for(i=0;i<n1;i++){
        L[i] = a[l + i];
    }
    for(j=0;j<n2;j++){
    R[j] = a[m + 1 + j];
    }
    i=0;
    j=0;
    k=l;


    while(i<n1&&j<n2) {
        if (L[i] <= R[j]) {
            a[k] = L[i];
            i++;
        } else {
            a[k] = R[j];
            j++;
        }
```

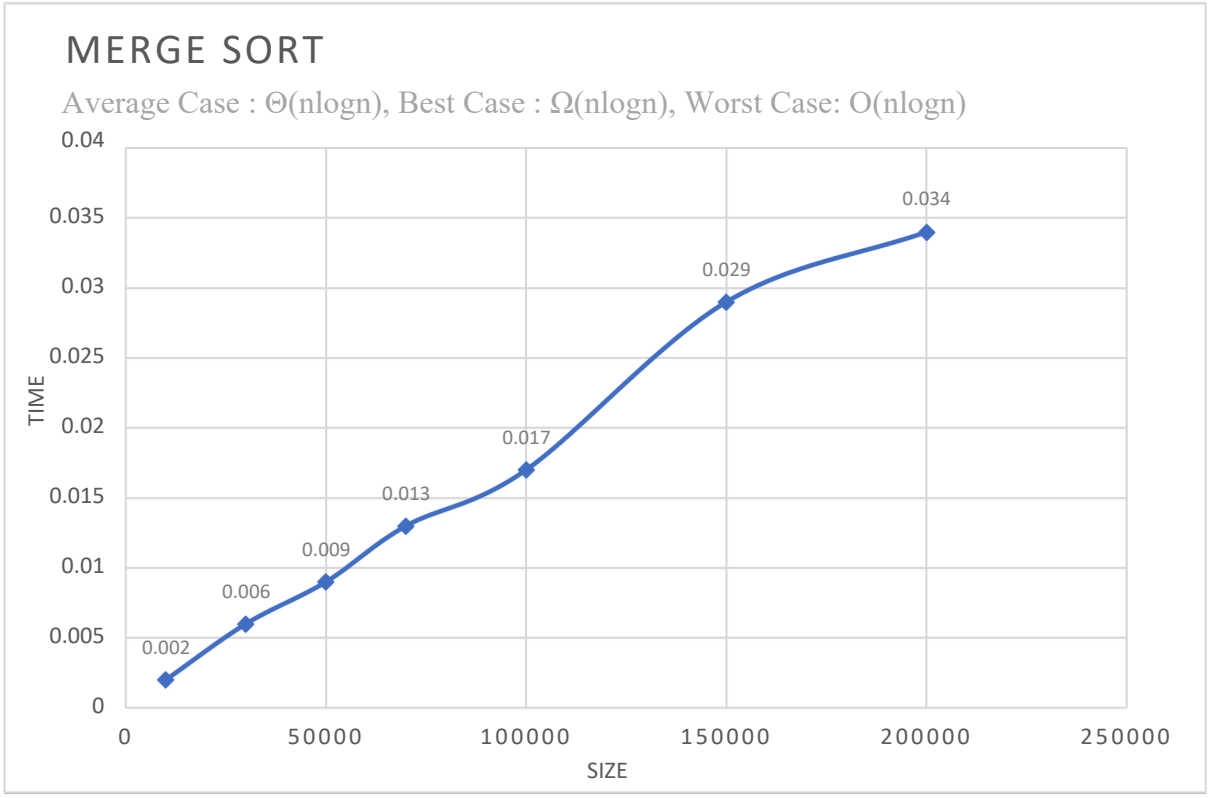
```
        k++;
    }
    while (i < n1) {
        a[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        a[k] = R[j];
        j++;
        k++;
    }
}

void merge_sort(int a[],int l,int r) {
    if(l<r){
        intm=l+(r-l)/2;
        merge_sort(a, l, m);
        merge_sort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int n;

    printf("Enter the number of elements: ");
    if(scanf("%d",&n)!=1||n<= 0) {
        printf("Invalidarraysize.\n");
```

```
        return 1;
    }

    intarr[n];

    printf("Enter %d elements:\n", n);
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    merge_sort(arr, 0, n - 1);
    printf("Sorted array: ");
    for(int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

Output:

```
Enter the number of elements: 6
Enter 6 elements:
10 7 12 8 20 15
Sorted array: 7 8 10 12 15 20
```

Graph:



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

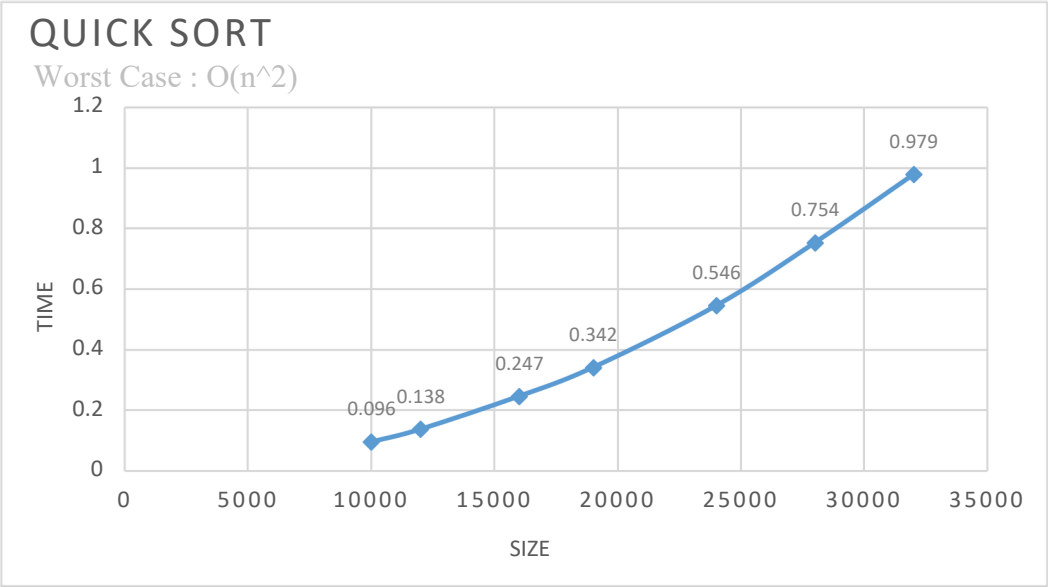
```
#include <stdio.h>


void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int a[], int low, int high) {
    int key = a[low];
    int i = low + 1;
    int j = high;
    while (1) {
        while (i <= high && a[i] <= key) {
            i++;
        }
        while (j >= low && a[j] > key) {
            j--;
        }
        if (i >= j) {
            swap(&a[low], &a[j]);
            break;
        }
        swap(&a[i], &a[j]);
    }
    return j;
```

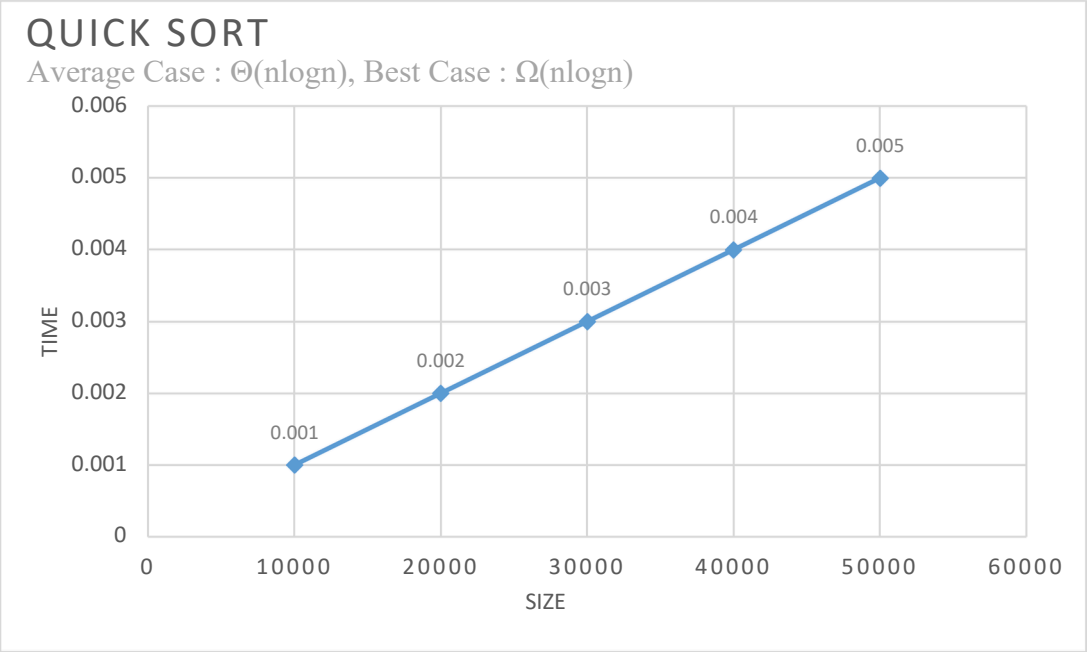
```
}
void quick sort(int a[], int low, int high) {
    if(low < high) {
        int j = partition(a, low, high);
        quicksort(a, low, j - 1);
        quicksort(a, j + 1, high);
    }
}
int main() {
    intn;
    printf("Enter the number of elements: ");
    if(scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }
    intarr[n];
    printf("Enter %d elements:\n", n);
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    quicksort(arr, 0, n - 1);
    printf("Sorted array: ");
    for(int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Output:

```
Enter the number of elements: 6
Enter 6 elements:
10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10
```

Graph:





Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

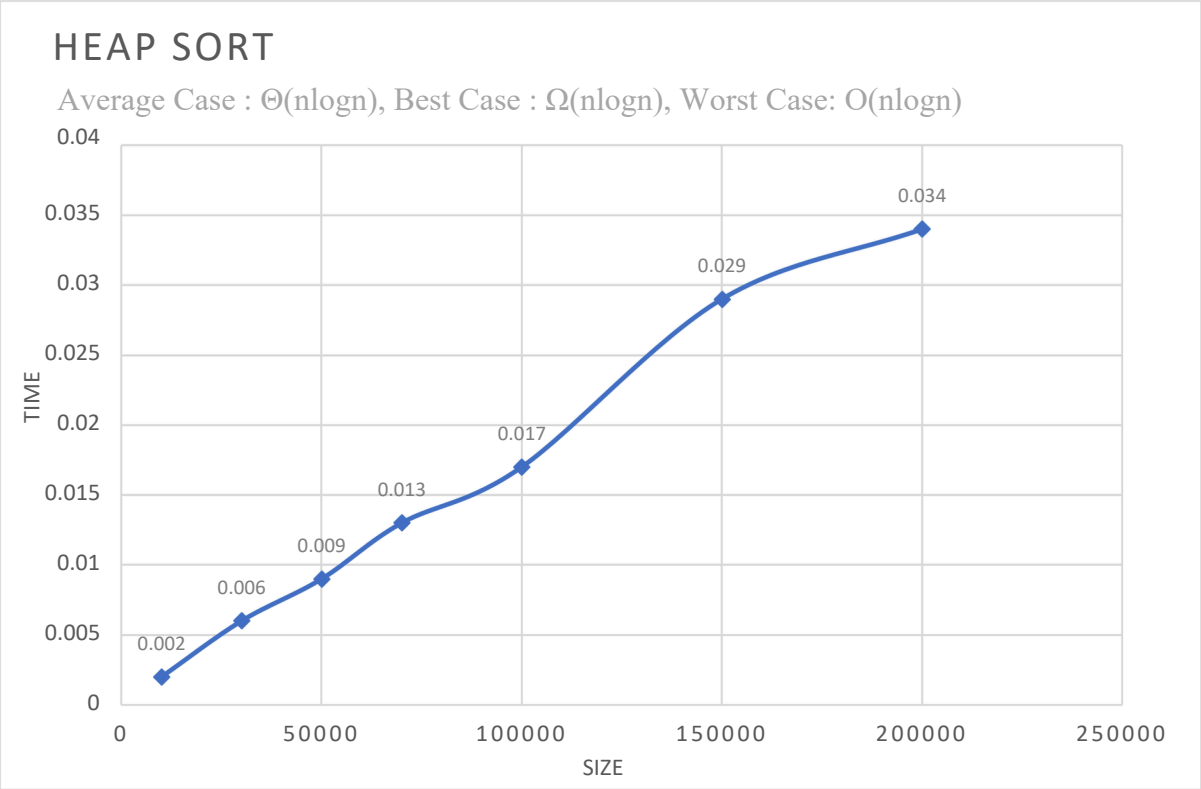
```
#include<stdio.h>


void heapify(int a[],int n, int p){
   intitem = a[p];
   intc= 2*p+1;
   while(c<=n-1){
      if(c+1<=n-1 && a[c+1] > a[c]) c++;
      if(item < a[c]){
         a[p] = a[c];
         p = c;
         c = 2*p+1;
      }
      else break;
   }
   a[p]= item;
}


void create_heap(int a[], int n){
   for(int i = (n-1)/2; i>=0; i--) heapify(a,n,i);
}


void heap_sort(int a[],int n){
   create_heap(a,n);
   for(int i = n-1; i>=0; i--){
      int temp = a[0];
```

```
        a[0] = a[i];
        a[i] = temp;
        heapify(a,i,0);
    }
}

void main(){
    intn;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    inta[n];
    printf("Enter the elements of the array :");
    for(int i = 0;i<n;i++){
        scanf("%d",&a[i]);
    }
    heap_sort(a,n);
    printf("Sorted: ");
    for(int i = 0;i<n;i++){
        printf("%d ",a[i]);
    }
}
```

Output:

```
Enter number of elements: 8
Enter the elements:
34 23 1 2 5 54 65 9

Unsorted Array:
34 23 1 2 5 54 65 9

Sorted Array:
1 2 5 9 23 34 54 65
```

Graph:


HEAP SORT
Average Case : Θ(nlogn), Best Case : Ω(nlogn), Worst Case: O(nlogn)
TIME
SIZE
0.04
0.035
0.03
0.025
0.02
0.015
0.01
0.005
0
0
50000
100000
150000
200000
250000
0.002
0.006
0.009
0.013
0.017
0.029
0.034

Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

Code:

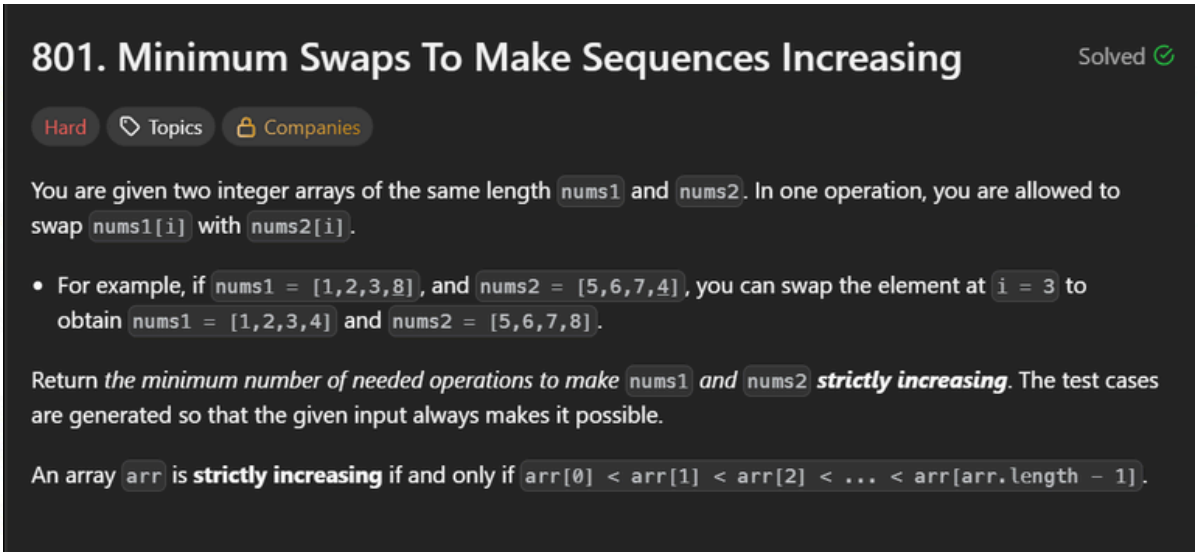
```
#include<stdio.h>


typedef struct item{
   int p;
   int w;
}p;


void main(){
int m,n;
printf("Enter maximum weight: ");
scanf("%d",&m);
printf("Enter the number of items");
scanf("%d",&n);
p pa[n+1];
for(int i=1; i<=n; i++){
      p p1;
      printf("Enter profit and weight: ");
      scanf("%d%d",&p1.p,&p1.w);
      pa[i] = p1;
   }
   int a[n+1][m+1];
   for(int i=0;i<=n;i++)
      a[i][0] = 0;
   for(int i=0;i<=m;i++)
      a[0][i] = 0;
```

```
    for(inti=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            if(j<pa[i].w)
                a[i][j] = a[i-1][j];
            else{
                int m = a[i-1][j];
                if(pa[i].p + a[i-1][j-pa[i].w] > m) m = pa[i].p + a[i-1][j-pa[i].w];
                a[i][j] = m;
            }
        }
    }
    printf("Maxiumum profit: %d",a[n][m]);
}
```

Output:

```
Enter the number of elements: 4
Enter the maximum capacity: 7
Enter the weights: 5
3
4
1
Enter the profits: 7
4
5
1
Total profit: 9

Process returned 0 (0x0)   execution time : 55.609 s
Press any key to continue.
```

LeetCode 801 - Minimum Swaps To Make Sequences Increasing:



801. Minimum Swaps To Make Sequences Increasing Solved

Hard | Topics | Companies

You are given two integer arrays of the same length `nums1` and `nums2`. In one operation, you are allowed to swap `nums1[i]` with `nums2[i]`.

- For example, if `nums1 = [1,2,3,8]`, and `nums2 = [5,6,7,4]`, you can swap the element at `i = 3` to obtain `nums1 = [1,2,3,4]` and `nums2 = [5,6,7,8]`.

Return *the minimum number of needed operations to make* `nums1` *and* `nums2` ***strictly increasing***. The test cases are generated so that the given input always makes it possible.

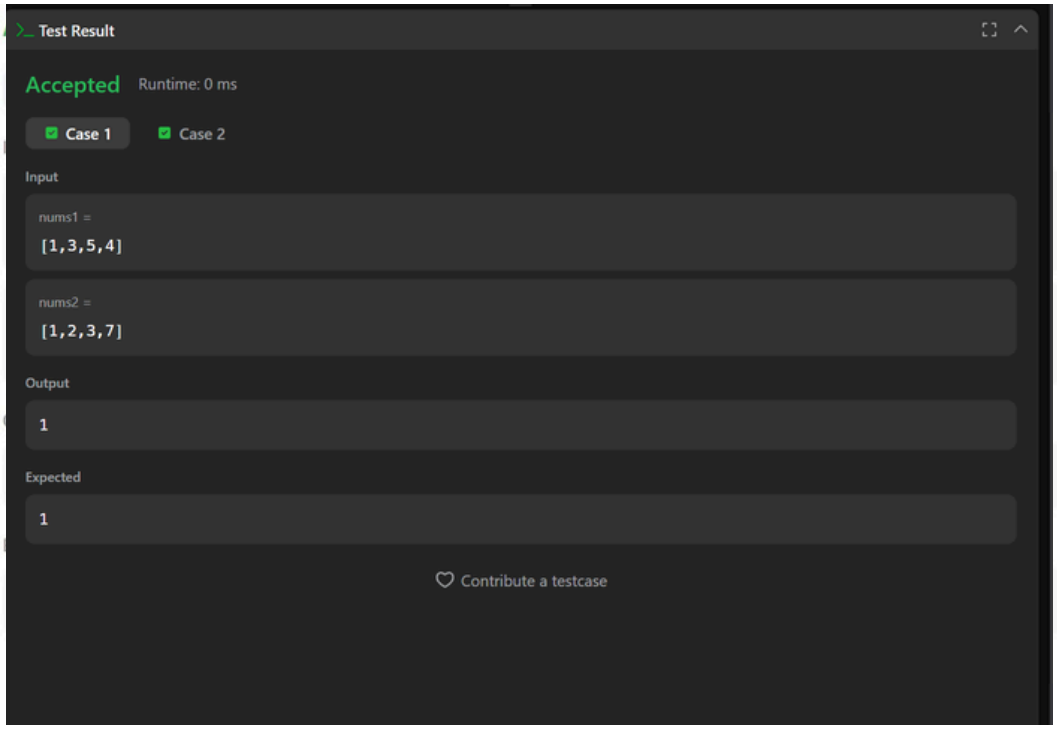
An array `arr` is **strictly increasing** if and only if `arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1]`.

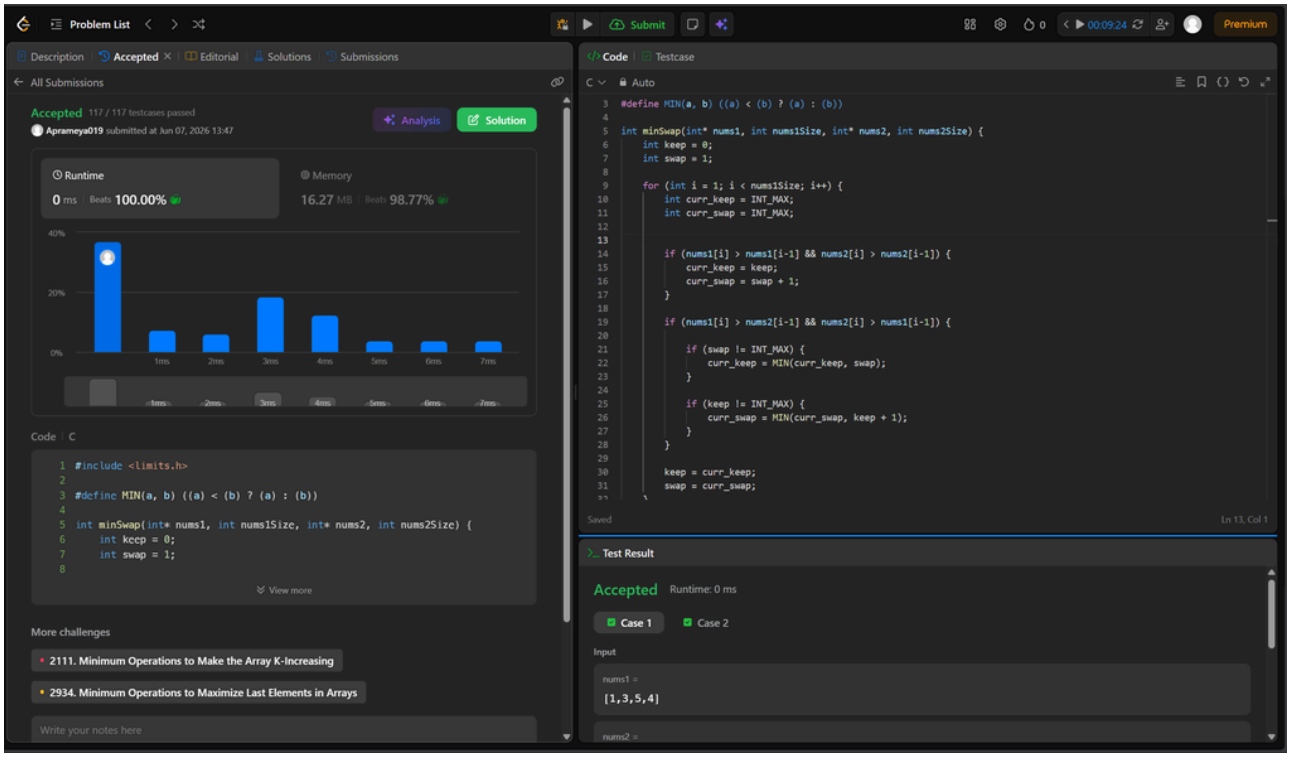
Code:

```
#define MIN(a, b) ((a) < (b) ? (a) : (b))

int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swap = 1;

    for (int i = 1; i < nums1Size; i++) {
        int curr_keep = INT_MAX;
        int curr_swap = INT_MAX;
            if (nums1[i] > nums1[i-1] && nums2[i] > nums2[i-1]) {
            curr_keep = keep;
            curr_swap = swap + 1;
        }
            if (nums1[i] > nums2[i-1] && nums2[i] > nums1[i-1]) {

            if (swap != INT_MAX) {
                curr_keep = MIN(curr_keep, swap);
            }

       if (keep != INT_MAX) {
       curr_swap = MIN(curr_swap, keep + 1);
       }
       }
```

```
keep = curr_keep;
swap = curr_swap;
}

return MIN(keep, swap);
}
```

Output:





Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

Code:

```
#include<stdio.h>


void floyds(int n,int a[n][n],int cost[n][n]){
    for(int i =0 ; i<n; i++){
        for(int j=0 ; j<n; j++){
            a[i][j] = cost[i][j];
        }
    }

    for(int k =0 ; k<n; k++){
        for(int i =0 ; i<n; i++){
            for(int j=0 ; j<n; j++){
                int min = a[i][j];
                min=(a[i][k]+a[k][j]<min) ? a[i][k]+a[k][j] : min;
                a[i][j] = min;
            }
        }
    }
}


void main(){
    intn;
 printf("Enter the number of vertices: ");
    scanf("%d",&n);
    int a[n][n], cost[n][n];
```

```
    printf("Enter the vertices: ");
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            scanf("%d",&cost[i][j]);
        }
    }
    floyds(n,a,cost);
    printf("Minimum path: ");
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            printf("%d ",a[i][j]);
        }
        printf("\n");
    }
}
```

Output:

```
Enter number of vertices: 4
Enter the cost matrix (use 999 for infinity):
0 999 6 1
4 0 20 10
999 3 0 12
6 999 999 0

The all-pairs shortest path matrix is:
0  9  6  1
4  0  10  5
7  3  0  8
6  15  12  0

Process returned 0 (0x0)   execution time : 54.621 s
Press any key to continue.
```

Lab Program 8:

a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

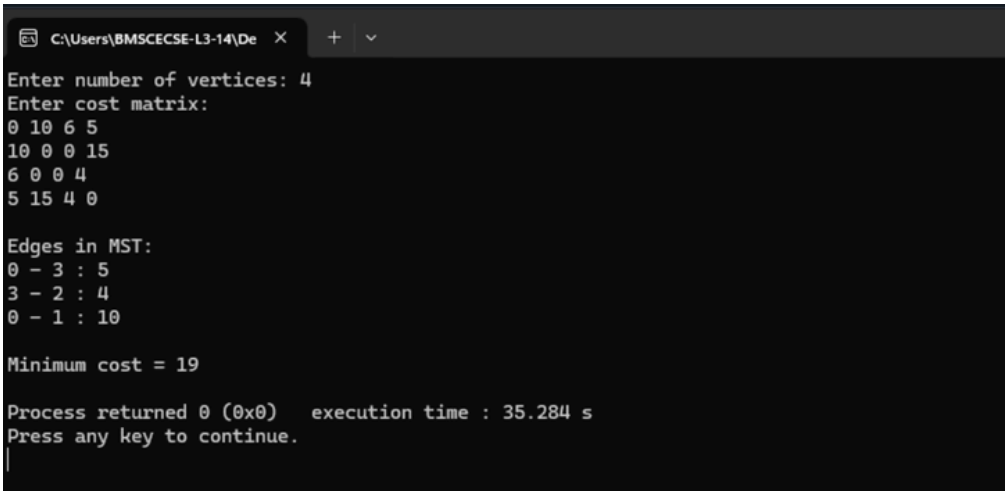
Code:

```
#include <stdio.h>


void prims(int n,int a[n+1][n+1]) {
   int near[n+1];
   int t[n][2];
   int mincost = 0;


   int min = 1000;
   int k = -1, l = -1;
   for(inti=1;i<=n; i++) {
      for(intj=i;j<= n; j++) {
         if(a[i][j]<min) {
         min=a[i][j];
         k=i;
         l=j;
         }
      }
   }


   t[0][0] = k;
   t[0][1] = l;
   mincost = a[k][l];


   for(inti=1;i<=n; i++) {
      if(a[i][k]<a[i][l]) near[i] = k;
```

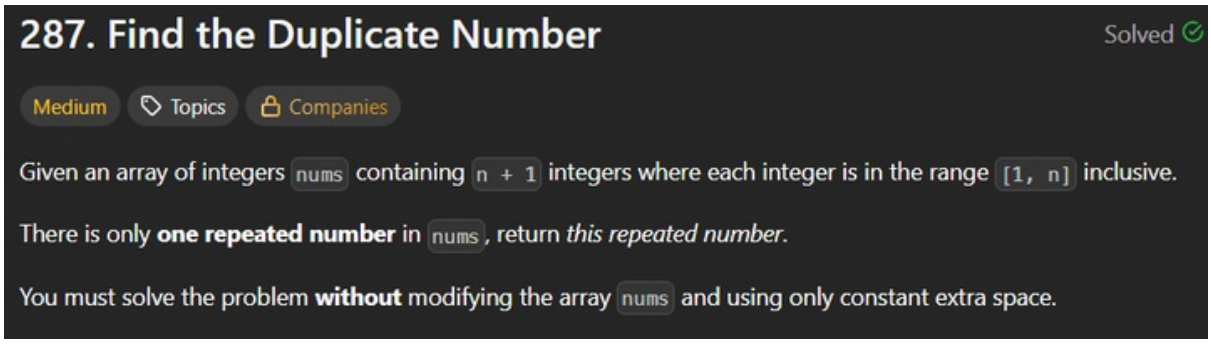
```
    else near[i] = l;
  }
  near[k] = near[l] = 0;

  for(inti=1;i<=n-2;i++) {
    int min = 1000;
    int j = -1;
    for(intm=1;m<=n; m++) {
      if(near[m]!=0&& a[m][near[m]] < min) {
        min=a[m][near[m]];
        j=m;
      }
    }

    t[i][0] = j;
    t[i][1] = near[j];
    mincost+=a[j][near[j]];
    near[j] = 0;

    for(intm=1;m<=n; m++) {
      if(near[m]!=0&& a[m][j] < a[m][near[m]]) {
        near[m] = j;
      }
    }
  }

  printf("EdgesinMST:\n");
  for(inti=0;i<n-1;i++) {
    printf("%d-%d\n",t[i][0], t[i][1]);
```

```
    }
    printf("Minimumcost = %d\n", mincost);
}

int main() {
    int n;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    int a[n+1][n+1];
    printf("Enter cost matrix:\n");
    for(inti=1;i<=n; i++) {
          for(intj=1;j<= n; j++) {
           scanf("%d",&a[i][j]);
       }
    }
    prims(n, a);
    return 0;
}
```

Output:



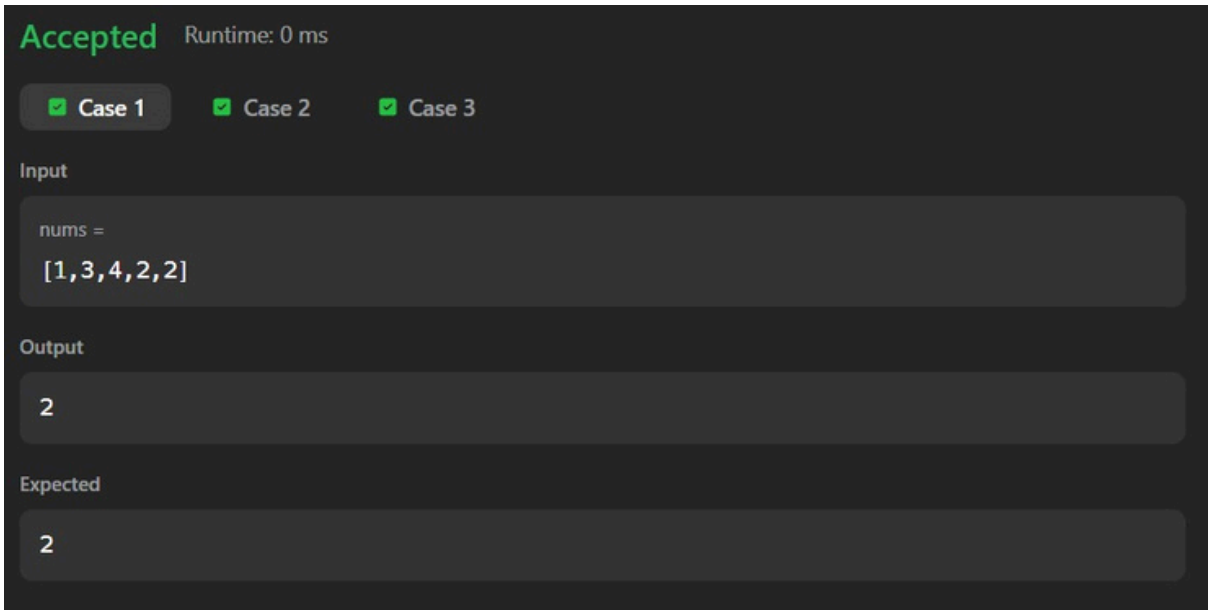
LeetCode 287 – Find The Duplicate Number:

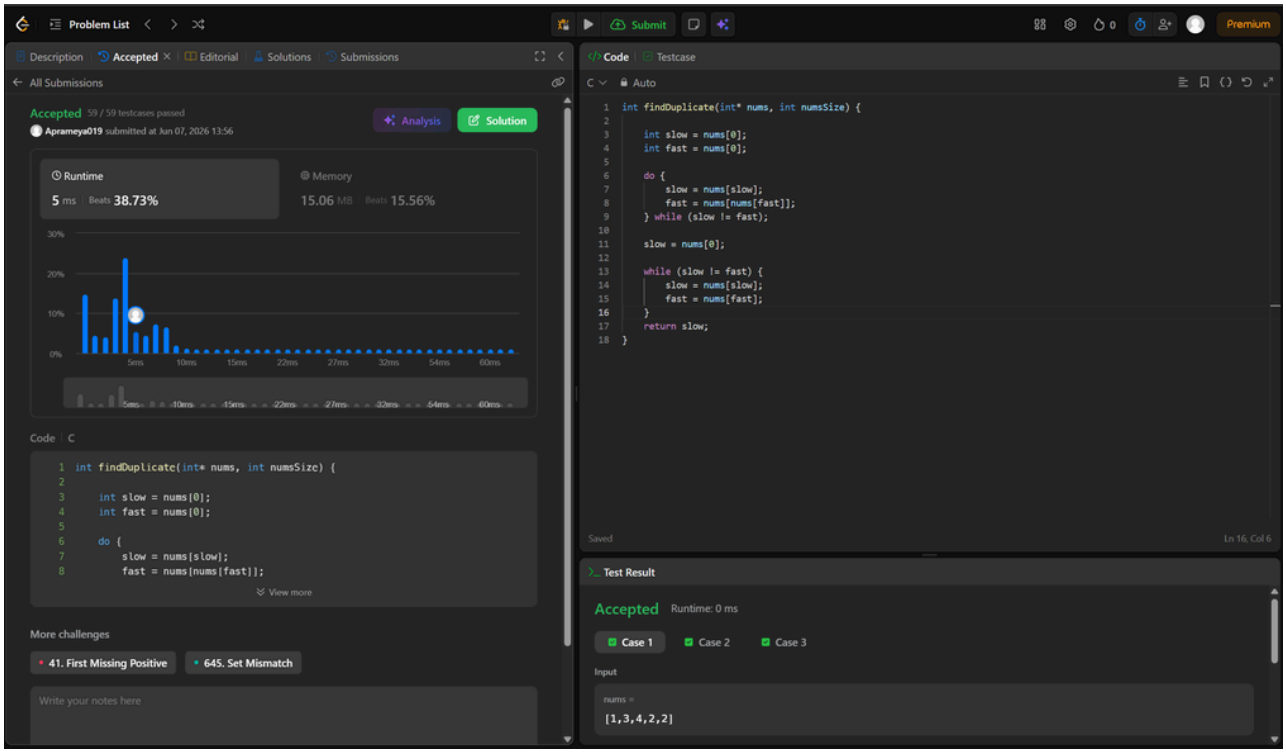


Code:

```
intfindDuplicate(int* nums, int numsSize) {
    int slow=nums[0];
    int fast=nums[0];
    do{
        slow=nums[slow];
        fast=nums[nums[fast]];
    }while(slow!=fast);
    slow=nums[0];
    while(slow!=fast){
        slow=nums[slow];
        fast=nums[fast];
    }
    return slow;
}
```

Output:


Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,3,4,2,2]
Output
2
Expected
2


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 59 / 59 testcases passed
Aprameya019 submitted at Jun 07, 2026 13:56
Analysis
Solution
Runtime
5 ms Beats 38.73%
Memory
15.06 MB Beats 15.56%
Code
C
int findDuplicate(int* nums, int numsSize) {
int slow = nums[0];
int fast = nums[0];
do {
slow = nums[slow];
fast = nums[nums[fast]];
View more
More challenges
41. First Missing Positive
645. Set Mismatch
Write your notes here
Code
Testcase
Auto
int findDuplicate(int* nums, int numsSize) {
int slow = nums[0];
int fast = nums[0];
do {
slow = nums[slow];
fast = nums[nums[fast]];
} while (slow != fast);
slow = nums[0];
while (slow != fast) {
slow = nums[slow];
fast = nums[fast];
}
return slow;
}
Saved
Ln 16, Col 6
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,3,4,2,2]

b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code:

```
#include<stdio.h>

int cost[10][10], n, t[10][2], sum;
void kruskal(int cost[10][10], int n);
int find(int parent[10], int i);

int main(){
    int i, j;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix:\n");
    for(i=0;i<n; i++){
        for(j=0;j<n; j++){
            scanf("%d", &cost[i][j]);
        }
    }

    kruskal(cost, n);
    printf("Edges in the Minimum Spanning Tree:\n");
    for(i=0;i<n-1; i++){
        printf("%d %d\n", t[i][0], t[i][1]);
    }

    printf("Total cost of the Minimum Spanning Tree: %d\n", sum);

    return 0;
```

```
}

voidkruskal(intcost[10][10], int n){
   int min, u, v, count, k;
   int parent[10];
   int i, j;

   k = 0;
   sum = 0;

   for(i=0; i<n; i++){
      parent[i] = i;
   }
   count = 0;
   while(count < n-1){
      min = 999;

      for(i=0; i<n; i++){
         for(j=0; j<n; j++){
            if(cost[i][j]!=0&& cost[i][j] < min){
               min=cost[i][j];
               u=i;
               v=j;
            }
         }
      }

      introot_u=find(parent,      u);
      introot_v=find(parent, v);
```

```
        if(root_u != root_v){
            t[k][0] = u;
            t[k][1] = v;
            sum += min;
            parent[root_v] = root_u;
            k++;
            count++;
        }
        cost[u][v] = cost[v][u] = 999;
    }
}

intfind(int parent[10], int i){
    while(parent[i] != i){
        i=parent[i];
    }
    returni;
}
```

Output:

```
Enter number of vertices: 4

Enter cost matrix (use 0 or 999 for no edge):
0  10 6  5
10 0  0  15
6  0  0  4
5  15 4  0

Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum cost = 19
```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

Code:

```
#include<stdio.h>
#define MAX 1000

typedef struct prop{
   float   p;
   float   x;
   float  w;
   float r;
}pr;

void sort(pr a[],int n){
   for(inti=0;i<n-1;i++){
      int max_p = i;
      for(intj=i+1;j<n;j++){
         if(a[max_p].r<a[j].r){
            max_p = j;
         }
      }
      pr temp = a[i];
      a[i] = a[max_p];
      a[max_p] = temp;
   }
}

void main(){
   int n;
```

```
float m;
printf("Enter the number of fruits: ");
scanf("%d", &n);
printf("Enter total weight: ");
scanf("%f", &m);
pr a[MAX];
for(inti=0;i<n;i++){
   prj;
   printf("Entertheprofit and weight");
   scanf("%f%f",&j.p,&j.w);
   j.x = 0;
   j.r = (j.p/j.r);
   a[i] = j;
}
sort(a,n);
float tot_profit=0;
int i = 0;
float rem = m;
while( i< n) {
   if(rem<a[i].w)break;
   a[i].x = 1.0;
   tot_profit+=a[i].p;
   rem -= a[i].w;
   i++;
}
if( i< n) {
   floatfrac=rem/a[i].w;
   a[i].x = frac;
   tot_profit+=frac* a[i].p;
```

```
    }

    printf("Here the maximum profit is : %f",tot_profit);
}
```

Output:

```
Enter the number of fruits: 4
Enter total weight: 100
Enter the profit and weight100 25
Enter the profit and weight120 25
Enter the profit and weight140 30
Enter the profit and weight150 50
Here the maximum profit is : 420.000000
```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code:

```
#include <stdio.h>

#define MAX 100
#define MAX_DIST 10000

int main() {
   inta[MAX][MAX],S[MAX], dist[MAX], n;
   int src;

   printf("Enterthenumber of vertices: ");
   scanf("%d", &n);

   printf("Entertheadjacency matrix:\n");
   for(inti=0;i<n;i++) {
      for(intj=0;j<n;j++) {
         scanf("%d",&a[i][j]);
      }
   }

   for(inti=0;i<n;i++) {
      S[i] = 0;
      dist[i]=MAX_DIST;
   }

   printf("Enterthesrcvertex: ");
```

```
 scanf("%d", &src);
    dist[src] = 0;

    for(intcount=0;count < n - 1; count++) {
        int min = -1;

        for(intj=0;j<n;j++) {
            if(!S[j]&&(min== -1 || dist[j] < dist[min])) {
                min = j;
            }
        }

        S[min] = 1;

        for(intj=0;j<n;j++) {
            if(a[min][j]&&!S[j] && (dist[min] + a[min][j] < dist[j])) {
                dist[j]=dist[min] + a[min][j];
            }
        }
    }
    printf("Distanceforeach vertex from source:\n");
    for(intk=0;k<n;k++) {
        printf("%d",dist[k]);
    }
    printf("\n");

    return 0;
}
```

Output:

Connected Graph:

```
Enter the number of vertices: 5
Enter the adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter the src vertex: 0
Distance for each vertex from source:
0 10 50 30 60
```

Disconnected Graph:

```
Enter the number of vertices: 6
Enter the adjacency matrix:
0 10 0 60 0 0
0 0 40 0 0 0
20 0 0 0 0 0
0 20 10 0 0 0
0 0 0 0 0 20
0 0 0 0 0 0
Enter the src vertex: 0
Distance for each vertex from source:
0 10 50 60 10000 10000
```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int x[20], n, count = 0;

int PLACE(int k, int i)
{
   for (int j = 1; j < k; j++)
   {
      if ((x[j] == i) || abs(x[j] - i) == abs(j - k))
         return 0;
   }
   return 1;
}
void printSolution()
{
   count++;
   printf("\nSolution %d:\n", count);
   for (int i = 1; i <= n; i++)
   {
      for (int j = 1; j <= n; j++)
      {
         if (x[i] == j)
            printf(" Q ");
         else
```

```
            printf(".");
        }
        p r int f( "\ n") ;
    }
}

voidNQueens(intn)
{
    int k = 1;
    x[k] = 0;
    while (k > 0)
    {
        x[k]=x[k]+1;
        while((x[k]<= n) && !PLACE(k, x[k]))
        {
            x[k]=x[k]+ 1;
        }
        if (x[k] <= n)
        {
            if (k == n)
            {
                printSolution();
            }
            else
            {
                k++;
                x[k] = 0;
            }
        }
```

```
        else{
            k--;
        }
    }
}

int main()
{
    printf("Enter no of queens: ");
    scanf("%d", &n);
    NQueens(n);
    if(count== 0)
    {
        printf("No solution exists.\n");
    }
    return 0;
}
```

Output: